

TypeScript Foundations: Arrays and Collections

Arrays



Jon Friskics

Senior Software Development Author, Pluralsight

@jonfriskics@mastodon.social | www.pluralsight.com



JavaScript Arrays: Create an Array and Access Data

```
let cityNames = ["Orlando, FL", "Draper, UT", "Westlake, TX"];

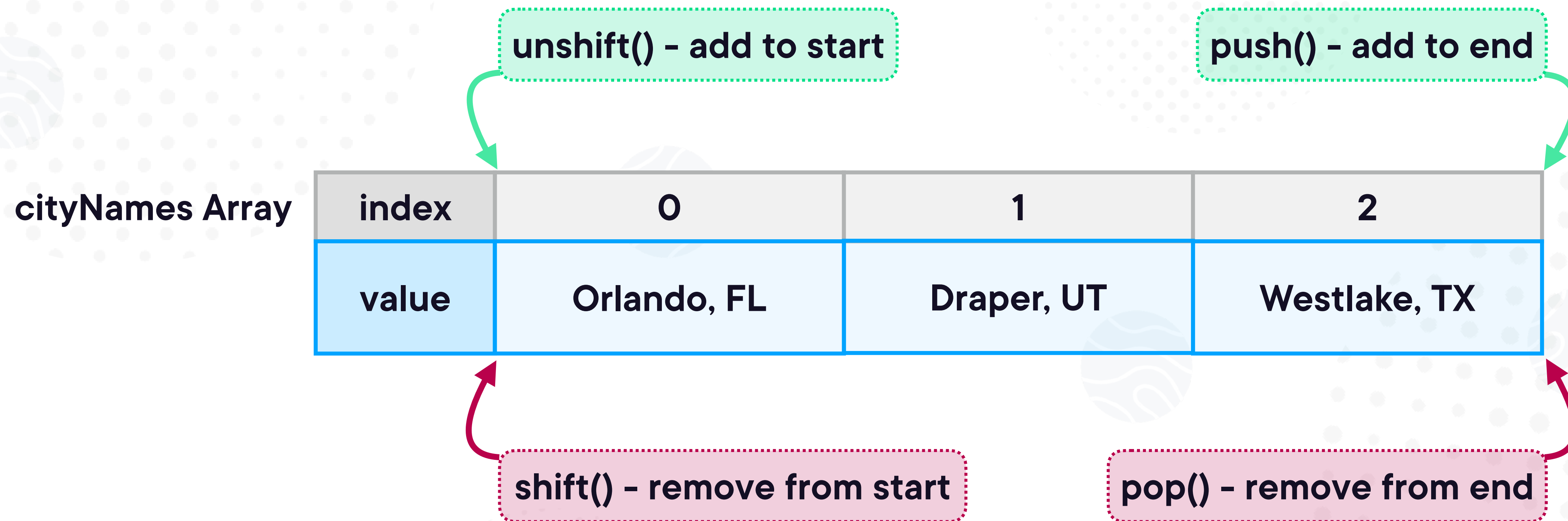
cityNames[0];    // "Orlando, FL"
cityNames[2];    // "Westlake, TX"
cityNames.length; // 3
```

cityNames Array

index	0	1	2
value	Orlando, FL	Draper, UT	Westlake, TX



JavaScript Arrays: Methods to Add/Remove Data



All four are mutating methods that modify the array they are called from

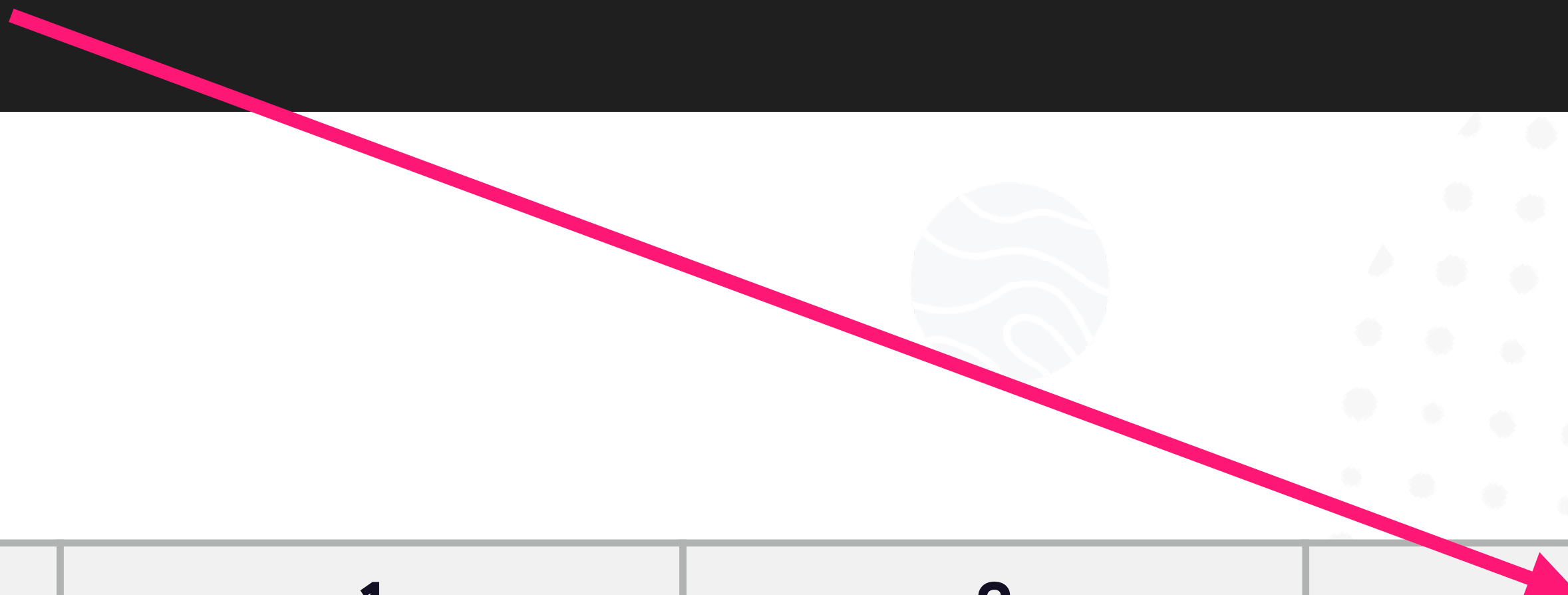


JavaScript Arrays: Add Elements with push()

```
let cityNames = ["Orlando, FL", "Draper, UT", "Westlake, TX"];  
  
let newLength = cityNames.push("San Francisco, CA");  
// newLength == 4
```

cityNames Array

index	0	1	2	3
value	Orlando, FL	Draper, UT	Westlake, TX	San Francisco, CA



JavaScript Arrays: Remove Elements with pop()

```
let cityNames = ["Orlando, FL", "Draper, UT", "Westlake, TX"];

let newLength = cityNames.push("San Francisco, CA");
// newLength == 4

let removedElement = cityNames.pop();
// removedElement === "San Francisco, CA"
```

cityNames Array

index	0	1	2	3
value	Orlando, FL	Draper, UT	Westlake, TX	San Francisco, CA



JavaScript Arrays: Combine with concat()

concat = concatenate, or "put things together"

```
let cityNames = ["Orlando, FL", "Draper, UT", "Westlake, TX"];  
  
let newCities = cityNames.concat("Atlanta, GA");
```

cityNames Array

index	0	1	2
value	Orlando, FL	Draper, UT	Westlake, TX

newCities Array

index	0	1	2	3
value	Orlando, FL	Draper, UT	Westlake, TX	Atlanta, GA



JavaScript Arrays: Iteration with forEach

```
let cityNames = ["Orlando, FL", "Draper, UT", "Westlake, TX"];

cityNames.forEach((city) => {
  console.log(city);
});

// logs each city on a new line:
// Orlando, FL
// Draper, UT
// Westlake, TX
```


JavaScript Arrays: Iteration with for...of

```
let cityNames = ["Orlando, FL", "Draper, UT", "Westlake, TX"];

for(let city of cityNames) {
  console.log(city);
};

// also logs each city on a new line:
// Orlando, FL
// Draper, UT
// Westlake, TX
```


JavaScript Arrays: Controlling Logic in for...of

```
let temperatures = [80, 85, 105, 70, 95];

for(let temperature of temperatures) {
  if(temperature < 90) {
    continue
  }
  console.log(`It's really hot today ${temperature}`);
};

// logs these 2 lines:
// It's really hot today 105
// It's really hot today 95
```

JavaScript Arrays: The map() method

```
let cities = ["orlando", "draper", "westlake"];  
let uppercaseCities = cities.map((city) => {  
    return city[0].toUpperCase() + city.slice(1);  
});  
  
// uppercaseCities == ["Orlando", "Draper", "Westlake"]
```



JavaScript Arrays: The filter() method

```
let temps = [78, 85, 105, 70, 50];  
let lowerTemps = temps.filter((temp) => temp < 80);  
  
// lowerTemps == [78, 70, 50]
```



JavaScript Arrays: Methods to Access Data

```
let cityNames = ["Orlando, FL", "Draper, UT", "Westlake, TX"];

cityNames.at(2);           // "Westlake, TX"
cityNames.at(-2);          // "Draper, UT"
cityNames.indexOf("Westlake, TX"); // 2
cityNames.includes("Westlake, TX"); // true
```

cityNames Array

index	0	1	2
value	Orlando, FL	Draper, UT	Westlake, TX



JavaScript Arrays: find() Method

```
let temperatures = [80, 85, 105, 70, 100];  
  
let t = temperatures.find((temperature) => temperature < 90);  
// t == 80
```

findIndex()
findLast()
findLastIndex()



JavaScript Arrays: sort() and toSorted()

```
let cityNames = ["Orlando, FL", "Draper, UT", "Westlake, TX"];

cityNames.sort();
// cityNames == ["Draper, UT", "Orlando, FL", "Westlake, TX"]

let temperatures = [80, 85, 105, 70, 100];
let sortedTemps = temperatures.toSorted();
// temperatures == [80, 85, 105, 70, 100];
// sortedTemps == [100, 105, 70, 80, 85];
```



JavaScript Arrays: Custom sort() Function

```
let temperatures = [80, 85, 105, 70, 100];  
let sortedTemps = temperatures.toSorted((a, b) => a - b);  
  
// temperatures == [80, 85, 105, 70, 100];  
// sortedTemps   == [70, 80, 85, 100, 105];
```


TypeScript Array Method Return Type Rules

If index is returned

return type is number

not found == -1

found == index number

If an element is returned

return type is type | undefined



Comparing TypeScript Collections

Collection Name	Arrays	Tuples	Sets	Maps
Order type	Ordered and can reorder			
How to declare	<code>type[]</code> or <code>Array<type></code>			
Index style	Positional index, starting at 0			
Duplicate values	Can contain duplicate values			
Length	Any length, can grow and shrink			
Sortable	Sortable			
Mutable	Mutable by default			
Immutable version	<code>readonly type[]</code> or <code>ReadonlyArray<type>()</code>			